

Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación

Programación III

TRABAJO PRÁCTICO FINAL

App World Cup 2026

Guía Técnica Versión 5.0 - 05-05-2026



Índice

- 1. Estructura del Proyecto**
- 2. WorldCupAPI: Endpoints Disponibles**
- 3. Notas Importantes**
- 4. Diagramas de Arquitectura**
 - 4.1. Arquitectura General de la Plataforma
 - 4.2. Detalle de Capas — Feature Plantel (Squad)
 - 4.3. Flujo Punta a Punta — "Plantel" desde ngOnInit() hasta el template

1. Estructura del Proyecto

El proyecto esta compuesto por tres piezas principales:

WorldCupAPI (Solo lectura - provista por la catedra)

Es la fuente de datos oficial del TP. Contiene la información de equipos, jugadores, entrenadores, simulaciones del mundial, partidos y la final. Los alumnos **NO la deben modificar ni agregar persistencia propia**; únicamente deben consumirla a través del WorldCupCore del equipo. Al momento de la evaluación, el proyecto será probado utilizando la WorldCupAPI provista en el repositorio de la cátedra. Si el proyecto no funciona con la WorldCupAPI provista por la cátedra **será motivo de desaprobación** del trabajo práctico.

Puerto por defecto: 5101

WorldCupCore (Backend - NestJS, desarrollado por los alumnos)

Es el Backend For Frontend (BFF). Expone una API REST consumida por la app Angular. Cada feature tiene su propio Controller y Service:

- Controller: recibe la petición HTTP y delega en el service.
- Service: obtiene datos de la WorldCupAPI, aplica lógica de negocio y devuelve el resultado al controller.
- Toda la lógica de negocio de las funcionalidades debe resolverse en el WorldCupCore.

Arquitectura: controller -> service -> WorldCupAPI

- Es de carácter **obligatorio** que el respete esta arquitectura a hora de desarrollar su **WorldCupCore**

Puerto por defecto: 4101

WorldCupApp (Frontend - Angular, desarrollado por los alumnos)

Aplicación Angular que consume la API del WorldCupCore. Cada feature tiene su propio componente y service:

- Componente: .html + .css + .ts. Muestra los datos al usuario.
- Service: se comunica **exclusivamente** con el WorldCupCore. WorldCupApp, debe hacer el menor procesamiento posible: solo adaptar la respuesta que le entrega WorldCupCore al modelo de vista del componente.
- Es de carácter **obligatorio** que el respete esta arquitectura a hora de desarrollar su **WorldCupApp**

Estructura: componente (.ts, .html, .css) + service + modelos (api.interface.ts / view-model.interface.ts)

2. WorldCupAPI: Endpoints Disponibles

A continuación se describen los endpoints de la WorldCupAPI que los alumnos pueden consultar desde el WorldCupCore. La API tiene documentación Swagger disponible en <http://localhost:5101/worldCup/api> una vez levantada. El WorldCupCore expone su propio Swagger en <http://localhost:4101/worldCupCore/api>.

Equipos - /teams

- GET /teams - Lista todos los equipos. Soporta filtros ?teamId= y ?name=
- GET /teams/coaches - Lista entrenadores. Filtros: ?teamId=, ?nationality=
- GET /teams/:teamId/players - Jugadores de un equipo (plantel completo).
- GET /teams/:teamId/stats - Estadísticas del equipo: ataque, defensa, mediocampo, overall, estrategia y formación actuales.
- GET /teams/:teamId/history - Historia del equipo: mundiales disputados y títulos ganados.
- GET /teams/:teamId/rivals - Rivales históricos del equipo.
- GET /teams/:teamId/strategy - Estrategia actualmente configurada.
- GET /teams/:teamId/formation - Formación actualmente configurada.

Mundial - /world-cup

- POST /world-cup/simulate - Simula el mundial completo hasta la final pendiente. Body: { teamId: string }. El WorldCupCore pasa el teamId leído del .env (TEAM_ID).
- GET /world-cup/current - Metadatos del mundial actual: status, finalistas, flags de ciclo de vida.
- GET /world-cup/current/groups - Tablas de posiciones de la fase de grupos del mundial actual.
- GET /world-cup/current/matches - Todos los partidos del mundial actual. Filtro opcional ?stage=
- GET /world-cup/current/stats - Estadísticas agregadas del torneo. Solo disponible cuando el mundial termino (status ENDED).
- GET /world-cup/current/player-stats - Estadísticas individuales por jugador. Filtro opcional ?teamId=
- GET /world-cup/current/awards - Premios del mundial (Balón de Oro, Bota de Oro, Fair Play, etc.). Solo cuando status es ENDED. Acepta ?lang=
- GET /world-cup/current/teams/:teamId/journey - Camino del equipo en el mundial actual. Acepta ?lang=
- GET /world-cup/history - IDs de todos los mundiales simulados, en orden cronológico inverso.
- GET /world-cup/:worldCupId - Metadatos de un mundial histórico por ID.
- GET /world-cup/:worldCupId/groups - Tablas de grupos de un mundial histórico.
- GET /world-cup/:worldCupId/matches - Partidos de un mundial histórico.
- GET /world-cup/:worldCupId/stats - Estadísticas de un mundial histórico.

- GET /world-cup/:worldCupId/awards - Premios de un mundial histórico. Acepta ?lang=
- GET /world-cup/:worldCupId/teams/:teamId/journey - Camino del equipo en un mundial histórico. Acepta ?lang=

Referencia - /reference

- GET /reference/game-dictionary?lang= - Diccionario del juego: posiciones (GK, DF, MF, FW), acciones, eventos, tipos de mensaje, estrategias y formaciones, con etiquetas y descripciones localizadas. Acepta ?lang=
- GET /reference/coach-profiles?lang= - Catálogo de perfiles de entrenador con etiqueta, descripción y ejemplo en idioma seleccionado. Útil para la feature de Cuerpo Técnico. Acepta ?lang=

Final / Partidos - /match

- POST /match/start-final - Inicia la final o retorna la final activa. Body: { teamId?: string }
- POST /match/play - Ejecuta un turno de la final. Body: { matchId: string, input?: string, messageType?: string }
- GET /match/current/stats - Estadísticas y timeline de eventos del partido activo. Acepta ?lang=
- GET /match/current/squad - Estado del plantel de ambos equipos en el partido activo (titulares, suplentes, tácticas).
- GET /match/current/lineups - Jugadores actualmente en cancha.
- GET /match/strategies - Catálogo de estrategias disponibles. Acepta ?lang=
- GET /match/formations - Catálogo de formaciones disponibles. Acepta ?lang=
- POST /match/select-strategy - Cambia la estrategia de un equipo. Body: { teamId: string, strategy: string }
- POST /match/select-formation - Cambia la formación de un equipo. Body: { teamId: string, formation: string }
- POST /match/reset-team-tactics - Restaura táctica por defecto del equipo. Body: { teamId: string }
- GET /match/historic-finals - IDs de finales históricas (excluye la final activa).
- GET /match/:matchId/stats - Estadísticas de un partido por ID.
- GET /match/:matchId/squad - Estado del plantel de un partido histórico.
- GET /match/:matchId/lineups - Jugadores en cancha de un partido histórico.
- GET /match/message-types - Lista de tipos de mensaje disponibles para las opciones de juego.

3. Notas Importantes

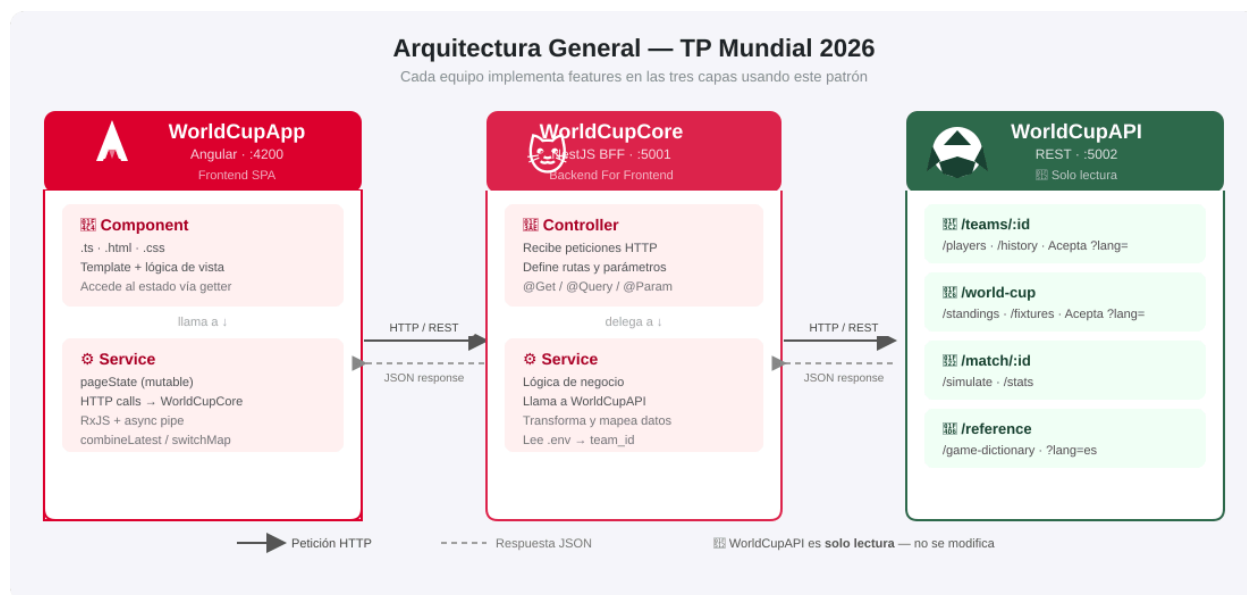
- Las funcionalidades provistas (Plantel, Historia del Equipo y Simular Final) son el modelo a seguir. **Antes de desarrollar tu funcionalidad, estudialas en profundidad.**
- El contrato de API entre WorldCupCore y WorldCupApp lo definen los alumnos: no está especificado en este enunciado. Es parte del ejercicio de diseño.
- La WorldCupAPI ya maneja la persistencia. El WorldCupCore no necesita base de datos propia ni se le debe agregar ninguna base de datos.
- El parámetro teamId utilizado por el WorldCupCore para identificar el equipo del usuario deberá configurarse mediante la variable de entorno TEAM_ID incluida en el archivo .env del proyecto esqueleto entregado a los alumnos.
- El idioma de las respuestas de la WorldCupAPI se controla mediante la variable de entorno APP_LANG definida en el .env del WorldCupCore (valor configurado: es). El WorldCupCore deberá leer dicha variable y enviarla como parámetro ?lang= en cada llamada a los endpoints de la WorldCupAPI que soporten internacionalización, para que las etiquetas y textos lleguen en español al frontend.
- Si la WorldCupAPI retorna un error 409 (Conflict) o 404, generalmente significa que no hay una simulación activa. El backend debe convertir ese error en una respuesta clara con un messageCode específico para que el frontend pueda mostrar el placeholder correspondiente.

4: Diagramas de Arquitectura

Los siguientes diagramas ilustran la arquitectura de la aplicación y el flujo de datos completo de la funcionalidad Plantel (Squad), que sirve como modelo de referencia para todas las features del TP.

4.1. Arquitectura General de la Plataforma

La app tiene tres capas. Cada feature sigue el mismo patrón: un Component y un Service en Angular, un Controller y un Service en NestJS, y uno o más endpoints de WorldCupAPI.



4.2. Detalle de Capas — Feature Plantel (Squad)

A continuación se detallan los componentes de cada capa para la feature Plantel, que sirve como referencia de implementación.

WorldCupApp — Angular :3101 · apunta a WorldCupCore :4101	WorldCupCore — NestJS (BFF) :4101 · TEAM_ID y APP_LANG en el .env	WorldCupAPI — Solo lectura :5101 · no modificar
SquadPageComponent • .ts + .html + .css • ngOnInit() → service.initialize() • Template lee pageState.xxx via getter • onSearchTermChange() • onPositionFilterChange()	SquadController • GET /my-team/players • Params: ?lang ?searchTerm ?position • Solo recibe y delega al Service • createOkResponse(result)	TeamsController • GET /teams/:teamId/players • Devuelve: SquadPlayerApiItem[] { name, position, shirtNumber, age, skill, attack, defense, energy, isCaptain }
SquadService (Angular) • pageState: SquadViewModel (objeto mutable) • getViewModel() devuelve pageState • initialize(): combineLatest([teamId\$, lang\$]) • setSearchTerm() / setPositionFilter() • loadPlayers(): GET → tap() muta pageState	SquadService (NestJS) • Lee teamId del .env via AdminService • Hace 2 llamadas a WorldCupAPI: ① GET /teams/:id/players ② GET /reference/game-dictionary • Ordena: GK → DF → MF → FW + shirtNumber • Construye positionLegend[] • Aplica filtros, calcula averageAge • Retorna SquadPlayersModel	ReferenceController • GET /reference/game-dictionary • Acepta ?lang=es • Devuelve: GameDictionaryApiResponse { positions: [{code, label, description}] }

4.3. Flujo Punta a Punta — "Plantel" desde ngOnInit() hasta el template

El siguiente diagrama muestra el recorrido completo de un request desde que el usuario abre la pantalla hasta que los datos se renderizan en la vista.

Angular :3101	NestJS :4101	WorldCupAPI :5101
① ngOnInit() Llama a <code>squadService.initialize()</code>		
② initialize() activa combineLatest <code>combineLatest([teamId\$, lang\$])</code> Llama <code>loadPlayers("", 'ALL')</code> <code>pageState.loading = true</code>		
③ HTTP Request → GET :4101/worldCupCore/my-team/players?lang=es&position=ALL	④ Controller recibe el request Delega a <code>SquadService.getPlayers()</code> Lee <code>teamId = 'arg'</code> del <code>.env</code> via <code>AdminService</code>	
	⑤ Hace 2 llamadas a WorldCupAPI →	⑥ WorldCupAPI recibe y responde GET /teams/arg/players?lang=es → <code>SquadPlayerApiItem[]</code> GET /reference/game-dictionary?lang=es → <code>GameDictionaryApiResponse</code>
	⑦ Service procesa los datos • Ordena jugadores: GK → DF → MF → FW • Construye <code>positionLegend[]</code> del diccionario • Aplica filtros de búsqueda y posición • Calcula <code>averageAge</code> → Retorna <code>SquadPlayersModel</code> al Controller	
⑨ Service recibe la respuesta <code>tap()</code> muta <code>pageState</code> directamente: <code>pageState.players = [...]</code> <code>pageState.filteredPlayers = [...]</code> <code>pageState.averageAge = 26.4</code>	⑧ Controller responde → <code>createOkResponse(squadPlayersModel)</code>	

<code>pageState.loading = false</code>		
⑩ Angular change detection Getter <code>pageState()</code> en el Component devuelve el objeto <code>pageState</code> actualizado Template re-renderiza la grilla de jugadores		